■■ Awesome Low Level Design

A curated guide to LLD, Object-Oriented Design & Interview Preparation

■ 26.9k • ■ 6.5k • github.com/ashishps1/awesome-low-level-design

■ OOP Topics	■ Design Patterns	■ LLD Problems	■ Concurrency
7 fundamentals	23 patterns	32 problems	9 problems

Based on the open-source repository by Ashish Pratap Singh • algomaster.io

■ Table of Contents

1.	Start Here & How to Learn LLD	pg 2
2.	OOP Fundamentals	pg 4
3.	Class Relationships	pg 6
4.	Design Principles (SOLID, DRY, KISS, YAGNI)	pg 8
5.	Design Patterns (Creational · Structural · Behavioral)	pg 10
6.	UML Diagrams	pg 12
7.	Concurrency & Multi-threading	pg 14
8.	LLD Interview Problems	pg 16
9.	Concurrency Problems	pg 18
10.	Resources & Books	pg 20

■ Start Here

New to LLD? Start with these two essential reads:

- How to Learn LLD — algomaster.io/learn/lld/how-to-learn-lld
- 15 Must-Know OOP Concepts — algomaster.io/learn/lld/15-must-know-oop-concepts

Low Level Design (LLD) focuses on the detailed design of individual components within a system — how classes, objects, and their interactions are structured to implement a feature or service. It bridges the gap between high-level architecture and actual code.

■ OOP Fundamentals

■ Classes & Objects

A class is a blueprint; an object is a runtime instance. Classes define attributes (fields) and behaviours (methods). Objects hold state and respond to messages.

■ Enums

Enumerations represent a fixed set of named constants. Useful for status codes, directions, days of the week, and any finite domain.

■ Interfaces

An interface defines a contract — a set of method signatures without implementations. Classes that implement an interface guarantee they provide those behaviours.

■ Encapsulation

Bundle data (fields) and the operations on that data (methods) together inside a class, and restrict direct access via access modifiers (private, protected, public).

■ Abstraction

Hide complex implementation details and expose only the necessary interface to the user. Abstract classes and interfaces are the primary tools.

■ Inheritance

A child class inherits fields and methods from a parent class, enabling code reuse and hierarchical classification. Favour composition over inheritance when possible.

■ Polymorphism

The same method name behaves differently depending on the object it is called on. Achieved via method overriding (runtime) and method overloading (compile-time).

■ Class Relationships

Relationship	Keyword	Description
Association	has-a	General link between two classes. Neither owns the other.
Aggregation	has-a (weak)	One class contains another, but the child can exist independently.
Composition	has-a (strong)	Parent owns the child; child's lifetime depends on the parent.
Dependency	uses-a	One class uses another temporarily (e.g. method parameter).

■ Design Principles

■ DRY — Don't Repeat Yourself

Every piece of knowledge must have a single, unambiguous, authoritative representation within a system. Avoid duplicating logic; extract reusable functions or classes.

■ YAGNI — You Aren't Gonna Need It

Don't add functionality until it is necessary. Over-engineering adds complexity without delivering immediate value.

■ KISS — Keep It Simple, Stupid

Design should be as simple as possible. Simpler code is easier to read, test, and maintain. Resist the urge to be clever.

■ SOLID Principles

Five object-oriented design principles that together produce maintainable, extensible code:

S — Single Responsibility: A class should have only one reason to change; each class owns exactly one responsibility.

O — Open / Closed: Software entities should be open for extension but closed for modification.

L — Liskov Substitution: Objects of a subclass must be substitutable for objects of the parent class without breaking correctness.

I — Interface Segregation: Clients should not be forced to depend on interfaces they do not use. Prefer many small interfaces.

D — Dependency Inversion: High-level modules should not depend on low-level modules. Both should depend on abstractions.

■ Design Patterns

Design patterns are reusable solutions to commonly occurring problems in software design. They are grouped into three categories:

Creational Patterns
• Singleton
• Factory Method
• Abstract Factory
• Builder
• Prototype

Structural Patterns
• Adapter

• Bridge
• Composite
• Decorator
• Facade
• Flyweight
• Proxy

Behavioral Patterns
• Iterator
• Observer
• Strategy
• Command
• State
• Template Method
• Visitor
• Mediator
• Memento
• Chain of Responsibility

■■ UML Diagrams

■ Class Diagram

Shows classes, their attributes, methods, and relationships. The most fundamental diagram for LLD interviews.

■ Use Case Diagram

Depicts actors and the system functions (use cases) they can perform. Useful for capturing functional requirements.

■ Sequence Diagram

Illustrates how objects interact in a particular time sequence. Shows the order of method calls between objects.

■ Activity Diagram

Represents workflows and business processes as a flowchart of activities. Useful for modeling complex business logic.

■ State Machine Diagram

Models the states of an object and transitions between them triggered by events. Essential for designing stateful systems.

■■ Concurrency & Multi-threading

Concurrency 101

- Introduction to Concurrency
- Concurrency vs Parallelism
- Processes vs Threads
- Thread Lifecycle and States
- Race Conditions and Critical Sections

Synchronization Primitives

- Mutex (Mutual Exclusion) — prevents simultaneous access to shared resources
- Semaphores — controls access to a limited number of resources
- Condition Variables — allow threads to wait until a condition is met
- Coarse-grained vs Fine-grained Locking
- Reentrant Locks — allow the same thread to acquire the lock multiple times
- Try-Lock and Timed Locking
- Compare-and-Swap (CAS) — lock-free atomic operation

Concurrency Challenges

- Deadlock — two or more threads wait on each other indefinitely
- Livelock — threads keep changing state in response to each other but make no progress

Concurrency Patterns

- Signaling Pattern
- Thread Pool Pattern
- Producer-Consumer Pattern
- Reader-Writer Pattern

■ How to Answer a LLD Interview Problem

1. Clarify Requirements	Ask clarifying questions to understand functional and non-functional requirements. Identify the scope: what to include and what to exclude.
2. Define Core Entities	Identify the main objects/classes in the system. Focus on nouns in the problem statement.
3. Identify Key Relationships	Determine how entities relate — association, aggregation, composition, inheritance.
4. Define Interfaces & Abstractions	Design interfaces before implementations. Apply SOLID principles.
5. Choose Design Patterns	Select appropriate design patterns (e.g. Observer for event-driven, Strategy for interchangeable algorithms, Factory for object creation).
6. Handle Edge Cases	Consider concurrency, error handling, and boundary conditions.
7. Draw Class Diagram	Sketch a UML class diagram showing classes, attributes, methods, and relationships.
8. Write Code	Implement the core classes with clean, readable, well-named code.

■ LLD Interview Problems

Easy Problems

• Design Parking Lot	• Design Stack Overflow
• Design a Vending Machine	• Design Logging Framework
• Design Traffic Signal Control System	• Design Coffee Vending Machine
• Design a Task Management System	

Medium Problems

• Design ATM	• Design LinkedIn
• Design LRU Cache	• Design Tic Tac Toe Game
• Design Pub Sub System	• Design an Elevator System
• Design Car Rental System	• Design an Online Auction System

• Design Hotel Management System	• Design a Digital Wallet Service
• Design Airline Management System	• Design a Library Management System
• Design a Social Network like Facebook	• Design Restaurant Management System
• Design a Concert Ticket Booking System	

Hard Problems

• Design CricInfo	• Design Splitwise
• Design Chess Game	• Design a Snake and Ladder game
• Design Ride-Sharing Service like Uber	• Design Course Registration System
• Design Movie Ticket Booking System	• Design Online Shopping System like Amazon
• Design Online Stock Brokerage System	• Design Music Streaming Service like Spotify
• Design Online Food Delivery Service like Swiggy	

■ Concurrency & Multi-threading Problems

- Print FooBar Alternately
- Print Zero Even Odd
- Fizz Buzz Multithreaded
- Building H2O Molecule
- Design Thread-Safe Cache with TTL
- Design Concurrent HashMap
- Design Thread-Safe Blocking Queue
- Design Concurrent Bloom Filter
- Multi-threaded Merge Sort

■ Books & Resources

Recommended Books

- Head First Design Patterns — Freeman & Robson
- Clean Code — Robert C. Martin
- Refactoring: Improving the Design of Existing Code — Martin Fowler

Online Courses

- Master LLD Interviews — algomaster.io/learn/lld/course-introduction
- Master Concurrency Interviews — algomaster.io/learn/concurrency-interview
- Coursera — Object-Oriented Design
- Coursera — Design Patterns

Newsletter

- AlgoMaster Newsletter — blog.algomaster.io

Generated from github.com/ashishps1/awesome-low-level-design • ■ 26.9k Stars • algomaster.io